

AN2DL 1st Challenge Report

Batch Size Matters

Matteo Rossi, Riccardo Piantoni, Jacopo Sacramone, Angelo Prete

matteoreds, plants, jsacri, angelopre

10798975, 10816545, 10752157, 10767149

November 17, 2025

1 Introduction

In this project, we address the automatic classification of patients' pain levels from sensor data collected during movements. Sensors placed on several joints record multivariate signals over time, and each patient is represented as a temporal sequence of feature vectors. From a Machine Learning perspective, this is a *Multivariate Time Series Classification* problem, where the goal is to assign each sequence to a discrete pain class. Our objective is to design an Artificial Neural Network that provides **reliable and accurate** pain predictions from limited, noisy data, evaluated using the F1 score. To this end, we follow an iterative development process. Starting from simpler models, our work focuses on:

- ① increasingly expressive architectures for sequential data, inspired by techniques discussed during the course and in the deep learning literature; and
- ② *preprocessing* steps that reduce dimensionality while preserving the most informative features.

2 Problem Analysis

The (*training*) dataset contains 661 patients, each observed over 160 consecutive time steps. Every step has 31 continuous joint measurements (`joint.00-joint.30`) and four aggregated pain indicators on a $[0-2]$ scale (`pain_survey_1-pain_survey_4`). The task is to

predict one of three pain categories: `no_pain`, `low_pain`, or `high_pain`. Each patient is also described by static attributes `n_legs`, `n_hands`, `n_eyes`, set to `one+x` for pirates, and to `two` for all others.

The main challenge is the limited information on the data acquisition process: combined with our limited medical background, this hinders interpretation of the numerical values and principled choices about dimensionality reduction without discarding potentially relevant information. In particular, we do not know the time unit, any temporal periodicity, or the spatial placement of the sensors, which makes it hard to group them by function or exploit anatomical symmetries. Moreover, the dimensionality of data for each patient is high while the number of available samples is limited, making it harder to learn complex patterns. We applied several techniques to tackle these issues, some of which led to improvements, while others did not.

Data exploration and preprocessing To limit the number of input features, we merged the static body-part attributes into a single boolean flag `is_pirate`. Inspecting the time series revealed that `joint_30` was constant across all measurements; therefore, it can be removed.

The training dataset contained a small number of clear outliers: patients 88, 465, and 472 exhibited pronounced spikes on specific joints at isolated time steps. While the high values of the `high_pain` pa-

tient (88) could in principle be informative, the spikes observed for the `low_pain` and `no_pain` patients (IDs 465 and 472) are likely due to sensor glitches or sudden movements, as they reach extreme values within one or two time steps and immediately return to their usual range, without any ramp-up or gradual decay.

To mitigate their impact, we detected and corrected them by examining local neighborhoods around suspicious peaks. For each anomalous segment, we compared the local maximum with the mean of neighbouring samples and, when inconsistent with the surrounding dynamics, applied *linear interpolation*, replacing outliers with the mean of adjacent time steps. Concretely, we corrected segments for patients 465 (joints `joint_21`, `joint_22`, `joint_25`, at time steps 130–131), and 472 (joint `joint_20`, at time step 27).

A Pearson correlation analysis highlighted strong dependencies between some joints, e.g. `joint_10` and `joint_11` (~ 0.94), as well as other correlated clusters, suggesting the possibility of dropping highly correlated joints.

3 Method

Our methodology combines a two-stage deep architecture with a carefully designed training-validation pipeline. A first-stage model extracts temporal representations and produces robust window-level predictions, which are aggregated into patient-level logit sequences; a second-stage *meta-model* then performs the final classification.

Models at both stages are evaluated using (Stratified) K-Fold Cross Validation, with particular care to avoid data leakage at every step of the pipeline, so that performance estimates are reliable. Finally, we perform coarse hyperparameter tuning with Optuna [1], balancing model performance with limited computational resources.

3.1 Final model architecture

Our final model is organized in two sequential stages. In the first stage, the network takes as input windows extracted from the preprocessed patient sequences. A 1D convolutional layer (1D-Conv) transforms raw joint signals at each timestamp into a higher-level feature sequence, which is then processed by a Bidirectional GRU (BiGRU) with an

Attention mechanism and a Multi-Layer Perceptron (MLP) head, producing window-level logits, i.e., class scores expressing the model’s confidence about the label of the original window.

The second-stage meta-model operates on the sequences of logits produced by the first-stage ensemble. For each patient, we collect the Out-Of-Fold (OOF) logits of all its windows, reorder them in time, and feed the resulting sequence to another BiGRU w/ Attention, trained with stronger regularization to limit overfitting. This meta-model learns how to weight and combine temporal information across windows, instead of relying on *naïve* averaging. In both stages, we adopt GELU [2] as the activation function, due to its smoothness and empirically better performance compared to ReLU. Training uses a class-weighted Cross-Entropy loss with label smoothing [3], where per-class weights emphasize under-represented classes and softened targets assign probability $1 - \varepsilon$ to the true class and distribute ε over the others, encouraging less overconfident and more robust predictions.

Table 1: Performance gains on 5 validation folds after applying modifications to the vanilla version of the model (*baseline*). Each row is an experiment comprising all techniques in the previous rows.

Addition	5-Fold Weighted F1
Two-Stage (<i>Baseline</i>)	0.9319 $\pm$ 0.0057
1. – Lab. Smoothing	(+ 0.0015) 0.9334
2. – 1+Weighted Loss	(– 0.0015) 0.9304
3. – 2+GELU	(– 0.0030) 0.9289
4. – 3+Sin. Encoding	(+ 0.0030) 0.9349
5. – 4+No Corr. Joints	(– 0.0061) 0.9258

4 Experiments

We evaluated several model architectures and training variants. For each configuration, we measured loss, accuracy, and weighted F1 on 5 validation folds, reporting mean and standard deviation. As baselines, we considered a classifier assigning the majority class for each sample and a random classifier sampling labels according to the empirical class priors. Since the 1st baseline requires no *training*, the performance was computed on the full dataset; for the random baseline, predictions were sampled five times to mimic a 5-fold setting. All results

Table 2: Comparison of the performance of different models on the respective validation sets.

Model	Accuracy (Micro F1)	Weighted F1
All <code>no_pain</code> labels	0.7731	0.2907
Sampling on training prior distribution	0.6452 ± 0.0575	0.6291 ± 0.0212
FeedForward (prediction on final window)	0.9298 ± 0.0057	0.9286 ± 0.0066
FeedForward + Windowing + Attention	0.9410 ± 0.0101	0.9466 ± 0.0054
BiGRU (w/windows)	0.9113 ± 0.0069	0.9113 ± 0.0056
Two-Stage Architecture (Kaggle Best)	0.9289 ± 0.0095	0.9377 ± 0.0106

are summarized in table 2. As the final architecture, we selected the model that achieved the best performance after applying all beneficial modifications. The individual impact (positive or negative) of each such modification is reported in table 1. We also tested data augmentation using Gaussian noise, learning rate schedulers, and Test-Time Augmentation, which showed no clear gains or degradations and were therefore not analyzed in detail.

5 Results

Our custom two-stage architecture was the best-performing configuration, achieving a **weighted F1 score of 0.9630 on the public Kaggle test set** and consistently leading in cross-validation. Label smoothing and class-weighted loss both improved performance by encouraging calibrated predictions and mitigating class imbalance, while switching from ReLU to GELU further boosted results. Windowing and explicitly encoding temporal seasonality also contributed positively. By contrast, dropping highly correlated joints, removing outliers, and applying gradient clipping brought no benefits and, in some cases, harmed performance, suggesting that our models were already handling redundancy, noise, and gradient stability effectively or possibly due to suboptimal implementations, choices, or interactions with the architecture.

6 Discussion

Our experiments show that, although deep models can learn rich representations from data, a carefully

designed architecture and preprocessing pipeline still provides clear benefits. Informed choices about features, staging, and training help the model focus on meaningful structure, while naïve modifications can degrade performance if they conflict with its inductive biases. A key strength of our final model lies in its features and processing capabilities. The 1D convolutional layer transforms raw joint-signals into higher-level representations, and naturally handles correlated inputs—explaining why manually dropping highly correlated joints was detrimental. A second strength comes from the second stage meta-model on sequences of window-level logits. By restoring the temporal order, it recovers information that simple averaging would discard, leading to more accurate predictions.

7 Conclusions

We framed automatic pain recognition from joint sensor data as a Multivariate Time Series Classification problem and proposed a two-stage deep architecture with 1D convolutions, BiGRUs with Attention, and a meta-model on window-level logits, achieving an **F1 of 0.9630** on Kaggle. Our results show that careful architectural design, windowing, seasonality encoding, label smoothing, and class-weighted loss are more beneficial than aggressive feature pruning or hyperparameter search. Future work could exploit richer information on the data acquisition process, explore alternative meta-models, complex architecture components such as Transformers, and investigate techniques to leverage limited data and class imbalance.

References

- [1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [2] D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus), 2023.
- [3] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision, 2015.